The result is the same collection of metrics you got from the `metrics` endpoint, but this time they are exported using a format understood by Prometheus. The following snippet shows an extract of the complete response, highlighting metrics related to the current number of threads:

```
# HELP jvm_threads_states_threads The current number of threads
# TYPE jvm_threads_states_threads gauge
jvm_threads_states_threads{state="terminated",} 0.0
jvm_threads_states_threads{state="blocked",} 0.0
jvm_threads_states_threads{state="waiting",} 13.0
jvm_threads_states_threads{state="timed-waiting",} 7.0
jvm_threads_states_threads{state="new",} 0.0
jvm_threads_states_threads{state="runnable",} 11.0
```

This format is based on plain text and is called *Prometheus exposition format*. Given the wide adoption of Prometheus for generating and exporting metrics, this format has been polished and standardized in OpenMetrics (https://openmetrics.io), a CNCF-incubating project. Spring Boot supports both the original Prometheus format (the default behavior) and OpenMetrics, depending on the `Accept` header of the HTTP request. If you'd like to get metrics according to the OpenMetrics format, you need to ask for it explicitly:

```
$ http :9001/actuator/prometheus \
    'Accept:application/openmetrics-text; version=1.0.0; charset=utf-8'
```

When you're done analyzing the Prometheus metrics, stop the application (Ctrl-C) and all the containers (`docker-compose down`).

NOTE You might encounter scenarios where you need to collect metrics from ephemeral applications or batch jobs that don't run long enough to be pulled. In that case, Spring Boot lets you adopt a push-based strategy so that the application itself sends metrics to the Prometheus server. The official documentation explains how to configure such behavior (http://spring.io/projects/spring-boot).

Spring Boot Actuator relies on the Micrometer instrumentation and provides auto-configuration to generate metrics for various technologies you might